



Testiranje i kvalitet softvera

III – Vrste testiranja

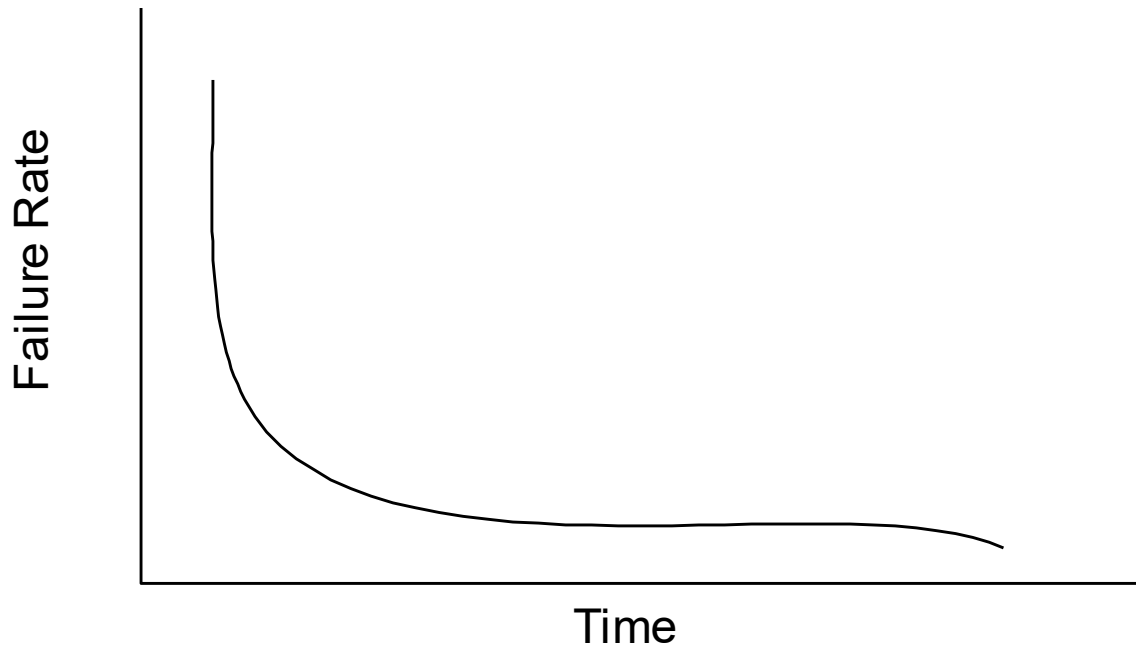
Prof. Dragan Janković
Katedra za Računarstvo

Sadržaj

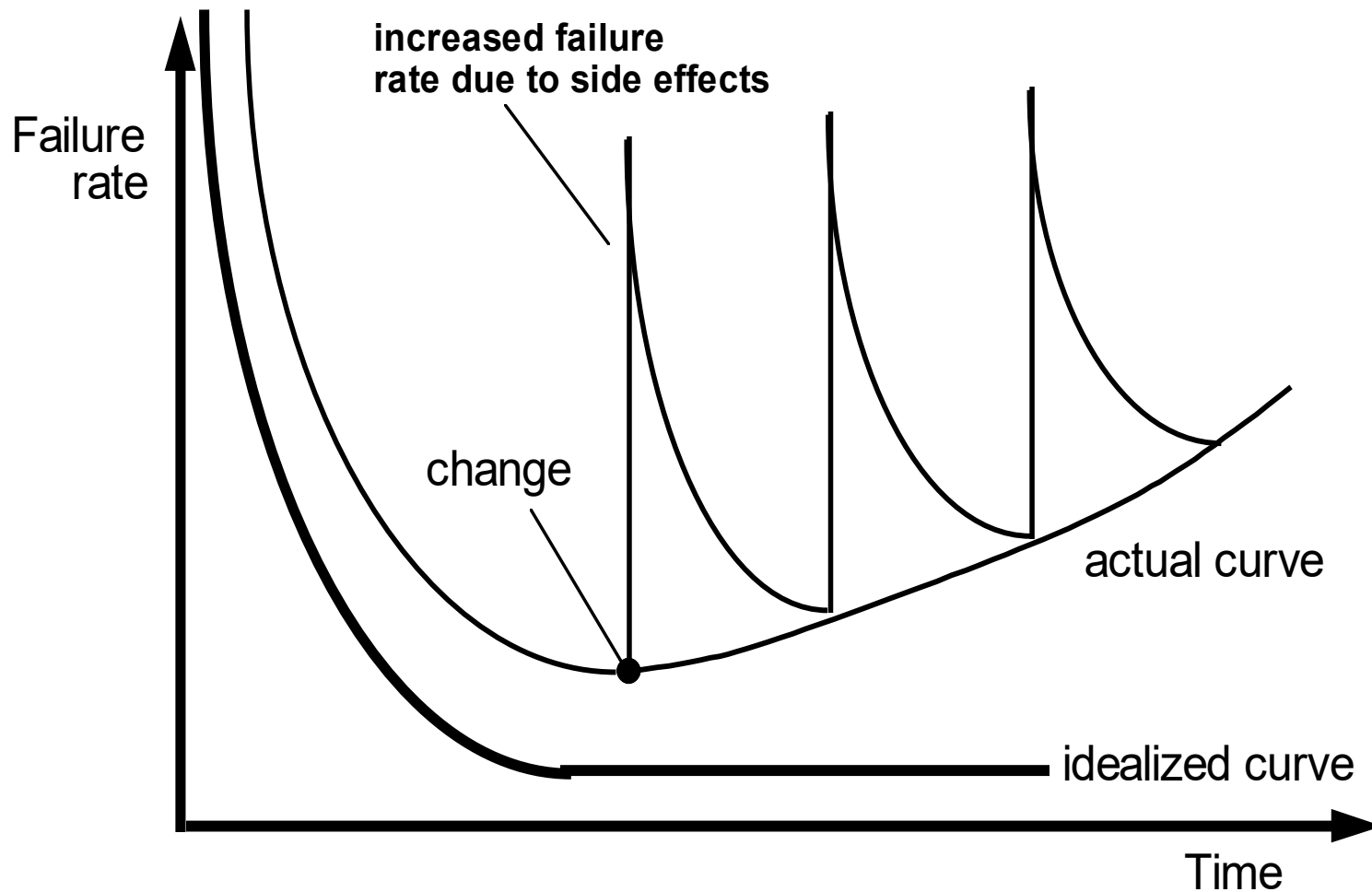
- Cena izmena
- Kvalitet softvera i testiranje
- Elementi testiranja
- Pozicija testiranja
- Klasifikacija testiranja



Raspodela Softverskih grešaka (idealna)

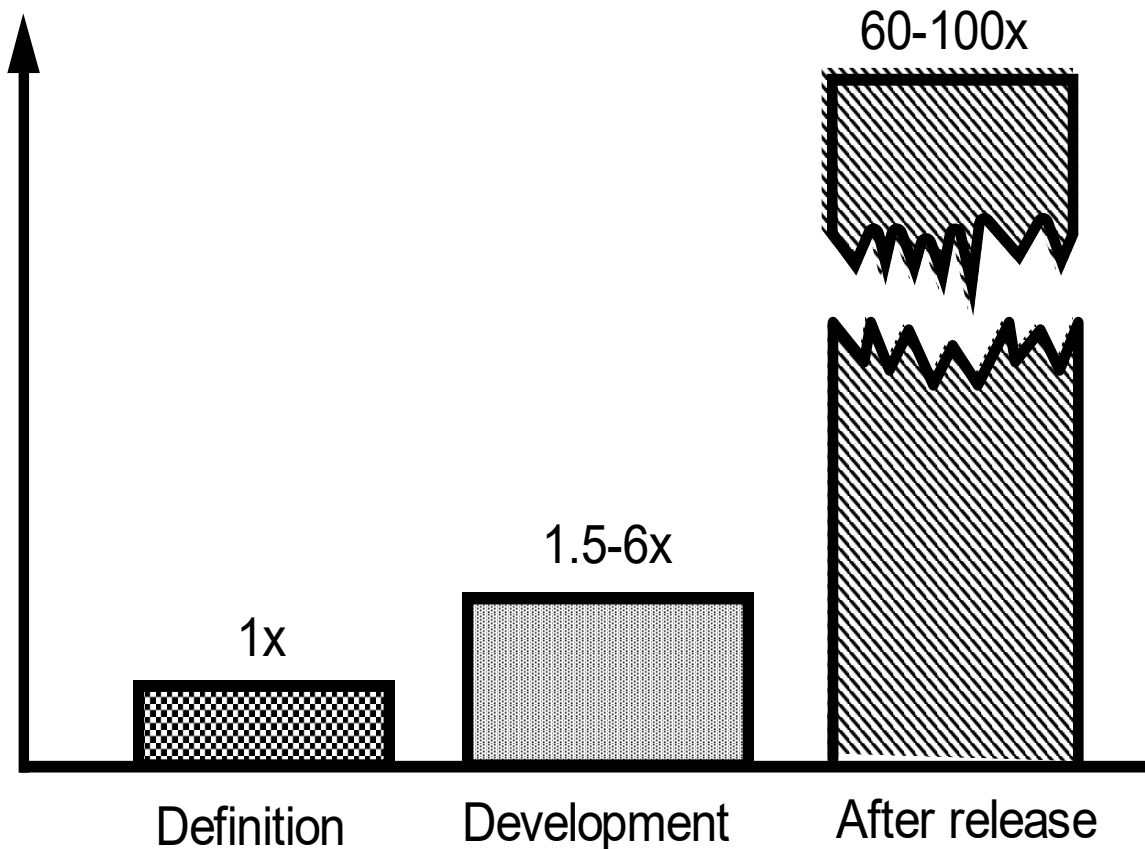


Raspodela Softverskih grešaka (realna)



Cena izmena

- Zavisi od trenutka kada se izmene vrše!



Kada treba da se vrši testiranje?

Greška pronadjena nakon isporuke košta četiri puta !!!(W. Perry)

- 1. trošak: razvoj programa pogrešan
- 2. trošak: sistem mora da se testira da bi se otkrila greška
- 3. trošak: pogrešna specifikacija ili kod treba da se uklone a korektna verzija da se doda tj. generiše
- 4. trošak: sistem mora ponovo da se testira!



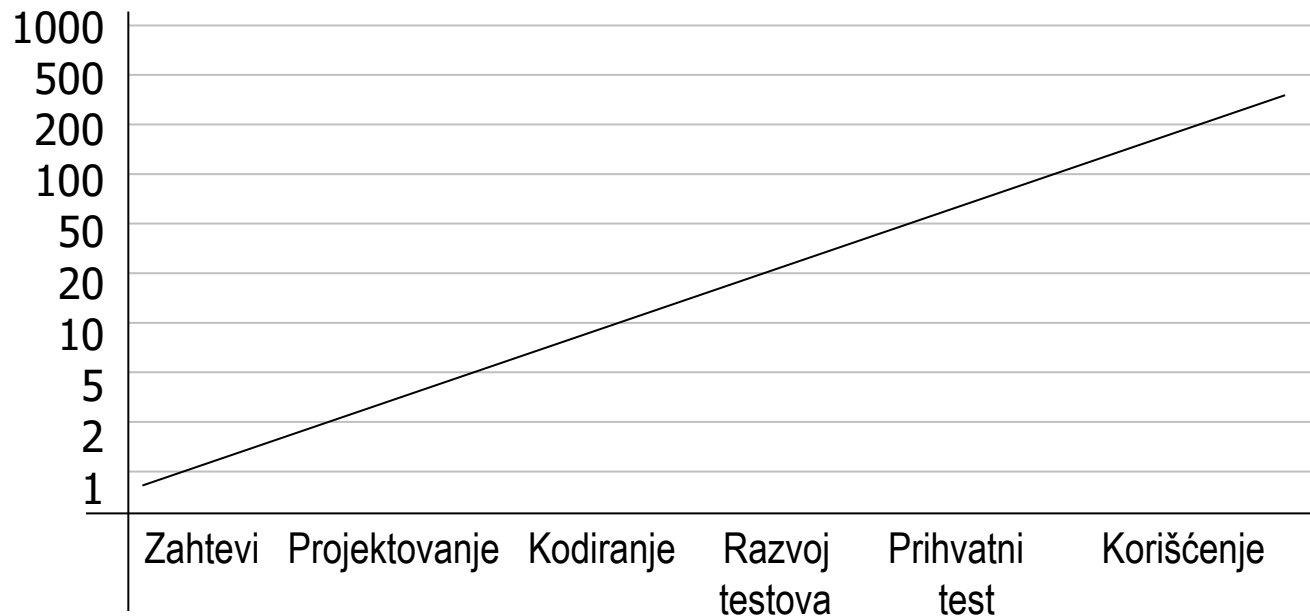
Vreme koje se troši na testiranje

- 50% Brooks/Myers, 1970s
- 80% Arthur Andersons' Director of testing in North America, 1990s
- 30% do 50% ukupnog napora (troška) je na testiranje
- Zahtevi za planiranim razvojem po fazama, dokumentovanjem, pridržavanjem raznih standarda, ispunjavanjem različitih nefunkcionalnih zahteva (sigurnost, portabilnost, performanse)
- prosečna produktivnost po osobi u celokupnom ciklusu razvoja industrijskog softvera je 300 do 1000 LOC/mes (LOC = linija izvornog koda)



Praksa korekcije grešaka

- Cena korekcije grešaka raste sa etapom u životnom ciklusu u kojoj je greška detektovana
- 60% grešaka se napravi u fazi projektovanja, 40% u toku implementacije
- 2/3 grešaka u projektovanju se otkrivaju tek kada je softver operativan !



Kvalitet softvera

■ Neposredni tj. direktni

- ☐ Korektnost
- ☐ Robusnost
- ☐ Sigurnost i pouzdanost
- ☐ Jednostavan za korišćenje
- ☐ Lak za učenje
- ☐ Efikasan

■ Dugoročni

- ☐ Proširiv
- ☐ Moguće ga koristiti ponovo (Reusability)
- ☐ Portabilan



Kvalitet

- Kvalitet je odsustvo “nedostataka” (ili “bugova”)
- (IEEE def.) Nedostaci rezultuju iz neke mane izazvane nekim propustom.
- Greška, mana, (Error) – u slučaju nedostatka, procena odstupanja od očekivanog rezultata.
- Nedostatak (Failure) – je bilo koji događaj u izvršenju sistema koji narušava neki od kriterijuma kvaliteta



Primer: Failure-Fault-Mistake

■ Y2K problem

- **Failure**: godine osoba se javljaju kao negativne!
- **Fault**: kod za računanje godina daje negativnu vrednost ako je datum rođenja u 20om veku a tekući datum je u 21om veku.
- **Mistake**: greška u računanju za datume nakon 20og veka



Zašto softver ima greške?

- Mi smo napravili neki propust
 - ☐ Nejasni zahtevi
 - ☐ Loše pretpostavke
 - ☐ Greške u projektovanju
 - ☐ Implementacione greške
- Neki aspekti sistema su teški za predviđanje
 - ☐ Kod velikih sistema, niko ne razume ceo sistem
 - ☐ Neka ponašanja je teško predvideti
 - ☐ Izuzetno velika složenost
- Evidencija
 - ☐ Česte greške usled postojanja “programiranja sa n-verzija”



Obezbeđenje kvaliteta (Kada – Ko?)

- Stalno - u toku celog razvojnog ciklusa
 - Pre – ugradjen u proces:
 - Proces (npr. CMMI, Agile, ...)
 - Metodologija (zahtevi, formalne metode, paterni, ...)
 - Alati, jezici
 - Posle – verifikacija:
 - Testovi
 - Ostale statičke i dinamičke tehnike

■ Nivoi

- Izbegavanje grešaka
- Detekcija grešaka (verification)
- Otpornost na greške (Fault tolerance)



Verifikacija i validacija*

- **Verifikacija:** Proces evaluacije sistema ili komponente u cilju odredjivanja da li rezultati date faze zadovoljavaju uslove postavljene na početku te faze. (Uglavnom paper-based aktivnost koja zahteva potvrdu da svaki korak razvoja ispunjava zahteve definisane u prethodnom koraku.)
- **Validacija:** Proces evaluacije sistema ili komponente u toku ili na kraju procesa razvoja da bi se odredilo da li on zadovoljava specificirane zahteve. (Uglavnom potvrda da implementirani sistem/komponenta radi po specifikaciji.)

* *IEEE Standard Glossary of Software Engineering Terminology, 1994*



Verifikacija

■ **Statička** (bez izvršenja)

- ☐ Rivjui (čovek)
- ☐ Provera koda i korišćenje alata
- ☐ Statička analiza
- ☐ Dokazivanje ispravnosti

■ **Hibridna** (uglavnom statička)

- ☐ Model checking
- ☐ Apstraktna interpretacija
- ☐ Simboličko izvršavanje

■ **Dinamička** (mora da se izvršava)

- ☐ Testiranje



Uspešnost testiranja

- Namena testiranja je pronalaženje "bugova" (Preciznije: izazivanje grešaka koje se detektuju neočekivanim ponašanjem usled postojećih nedostataka)
- Test se može nazvati "uspešnim" ako zapravo "ne prodje"
- Test koji prolazi ništa ne govori (osim ako prethodno isti test nije prolazio) (*regresiono testiranje*)
- U principu testiranje uključuje ljude koji nisu developeri (ali i njih)
- Testiranje se zaustavlja kada se detektuje bag (ne obuhvata korekciju tj. *debugiranje*)



Kako testirati ?

- Potpuno testiranje nije moguće
 - ☐ test svih validnih ulaza
 - ☐ test svih nevalidnih ulaza
 - ☐ test svih promenjenih ulaza
 - ☐ test svih varijacija u vremenu ulaza
- Dokazivanje korektnosti programa je suviše teško
- ==> izabrati dovoljno mali, još uvek adekvatan, skup testova tj. slučajeva



Testiranje – okvirna procedura

1. Identifikovati deo softvera koji će biti testiran
2. Identifikovati interesantne ulazne vrednosti
3. Identifikovati očekivane rezultate (funkcionalne) i karakteristike izvršenja (ne-funkcionalni zahtevi)
4. Startovati softver za ulazne vrednosti
5. Porediti rezultate i očekivane vrednosti i karakteristike izvršenja programa



Elementi testiranja

■ **Element koji se testira** (Implementation Under Test)

Softverski (& eventualno hardverski) elementi koji se testiraju

■ **Test case** (test primer)

Precizna specifikacija izvršenja kojom se pokriva moguća greška

Test primer je trojka [I, S, O] gde

- ☐ I predstavlja ulazne podatke
- ☐ S je stanje sistema u kome će podaci biti uneti
- ☐ O je **očekivani** izlaz

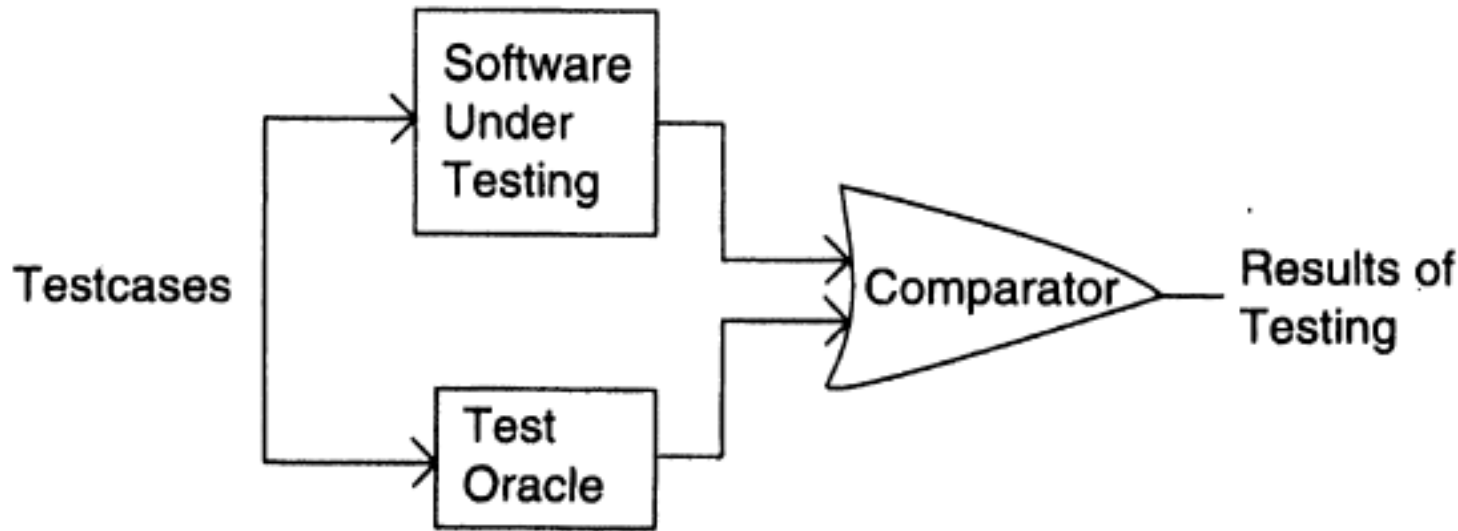
■ **Test run** - Jedno izvršenje test case-a

■ **Seriya testova** - skup svih test primera

- Test primeri se ne biraju na slučajan način. Umesto toga treba ih **projektovati**.



Očekivani rezultati testa



- **Predviđanje, predikcija rezultata testa (test oracle)**
 - Prediktor testa je mehanizam, nezavisan od samog programa, koji se može upotrebiti za proveru ispravnosti rada programa za test primere.
 - Konceptualno, test primeri se zadaju programu i prediktoru i njihovi izlazi potom međusobno upoređuju



Prediktor testa

- Može biti čovek ili automat
- Ljudi koriste specifikaciju programa da odluče šta je ispravno ponašanje programa.
Međutim, specifikacije programa su često sa greškama, nepotpune ili dvosmislene, a i ljudi mogu napraviti previd
- **Automatizovani prediktori** koriste formalne specifikacije i tačni su onoliko koliko je specifikacija tačna.
- Formalne specifikacije često ne postoje za program (nije ih lako napraviti)



Potreba za projektovanjem test primera

- Skoro svaki netrivialni sistem ima ekstremno veliki domen ulaznih podataka => iscrpno testiranje za svaku moguću kombinaciju ulaza je nemoguće ili nepraktično
- Slučajno izabrani test primer može biti bez značaja ako otkriva grešku koja je detektovana nekim drugim test primerom



Projektovanje test primera

- Broj test primera ne određuje koliko je testiranje delotvorno
- Da bismo uočili grešku u sledećem kodu
`if(x>y) max = x; else max = x;`
- Test 1: $\{(x=3, y=2); (x=2, y=3)\}$ zadovoljava
- Test 2: $\{(x=3, y=2); (x=4, y=3); (x=5, y = 1)\}$ ne zadovoljava
- Svaki test primer trebalo bi da razotkrije različitu grešku



Kriterijumi selekcije testova

- Kako da znamo koju ulaznu vrednost da uzmemo za testiranje, a koju ne?
- Mora se definisati **kriterijum selekcije**:
 - Za dati program P i njegovu specifikaciju S , kriterijum selekcije testova definiše uslove koje mora da zadovolji serija testova T .
 - Na primer, ako je kriterijum da se svaki iskaz programa mora izvršiti najmanje jednom, tada T zadovoljava ako za svaki iskaz u P postoji bar jedan test primer u T koji izaziva izvršavanje tog iskaza.



Osobine kriterijuma selekcije

- Osnovne osobine su **pouzdanost** i **validnost**
- Kriterijum C je **pouzdan** ako svaka serija testova (nije važno koju izaberemo) koja zadovoljava C detektuje iste greške.
- Kriterijum C je **validan** ako, za svaku grešku u programu, postoji neka serija testova koja zadovoljava C koja će otkriti grešku.
- **Ako je kriterijum C pouzdan i validan, a serija testova koja zadovoljava C prođe uspešno, onda program nema grešaka.**



Problemi sa kriterijumima selekcije

- Na žalost, nema načina da se za proizvoljan program identifikuje validan kriterijum.
- Kriterijum koji je validan i pouzdan, a da ga može zadovoljiti konačna serija testova, isto najčešće nije moguće naći.
- Zato se najčešće u realnosti definišu kriterijumi koji nisu ni pouzdani ni validni, kao što je “90% iskaza bi trebalo izvršiti bar jednom”.
- Za većinu kriterijuma nije moguće automatizovati generisanje testova koji zadovoljavaju kriterijum.
- *[Dijkstra] Testiranje programa može se koristiti da pokaže prisustvo grešaka u programu, nikada njihovo odsustvo.*
- Pošto se dakle sve greške u programu ne mogu garantovano otkriti testiranjem i uz navedena ograničenja kriterijuma, najčešće se u testiranju primenjuje neka kombinacija kriterijuma.
- Sve navedeno ima posledice na **odluku o završetku testiranja** o čemu će posebno biti reči kasnije



Elementi: Ocena testa

■ Očekivani rezultati (za test)

Precizna specifikacija šta se od testa očekuje ako nema grešaka:

- ☐ Povratne vrednosti
- ☐ Poruke
- ☐ Izuzeci
- ☐ Rezultujuće stanje program & okruženja
- ☐ Ne-funkcionalne karakteristike (vreme, memorija...)

■ Potvrda Testa

- ☐ Mehanizam kojim se određuje da li je izvršeni test zadovoljio očekivane rezultate
- ☐ Izlaz je po pravilu: prošao ("pass") ili nije prošao("fail").



Elementi: Izvršenje testa

■ Test driver

Program ili programski element (npr. klasa), korišćeni za primenu scenarija testiranja na jedan IUT

■ Stub

Privremena implementacija softverskog elementa, koja menja trenutnu implementaciju, u toku testiranja nekog drugog elementa.

U opštem slučaju ne zadovoljava potpunu specifikaciju elementa.

Može da posluži u slučajevima:

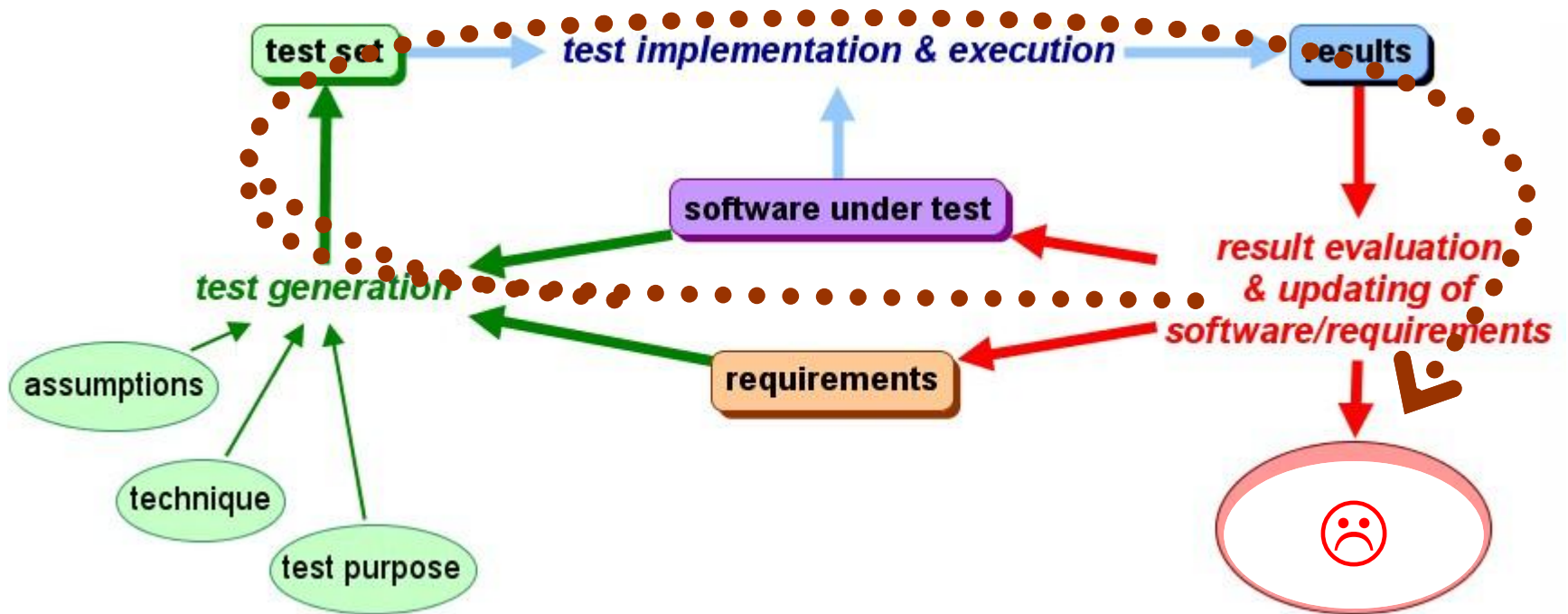
- ☐ Kada softverski element još nije napisan
- ☐ Externi softver koji ne može da se startuje u toku testa (npr. Zato što se zahteva pristup hardveru ili nekoj živoj bazi)
- ☐ Softverski element koji zahteva suviše mnogo vremena i memorije da bi se startovao i čiji rezultati mogu biti simulirani za potrebe testiranja.

■ Oprema/okruženje za testiranje

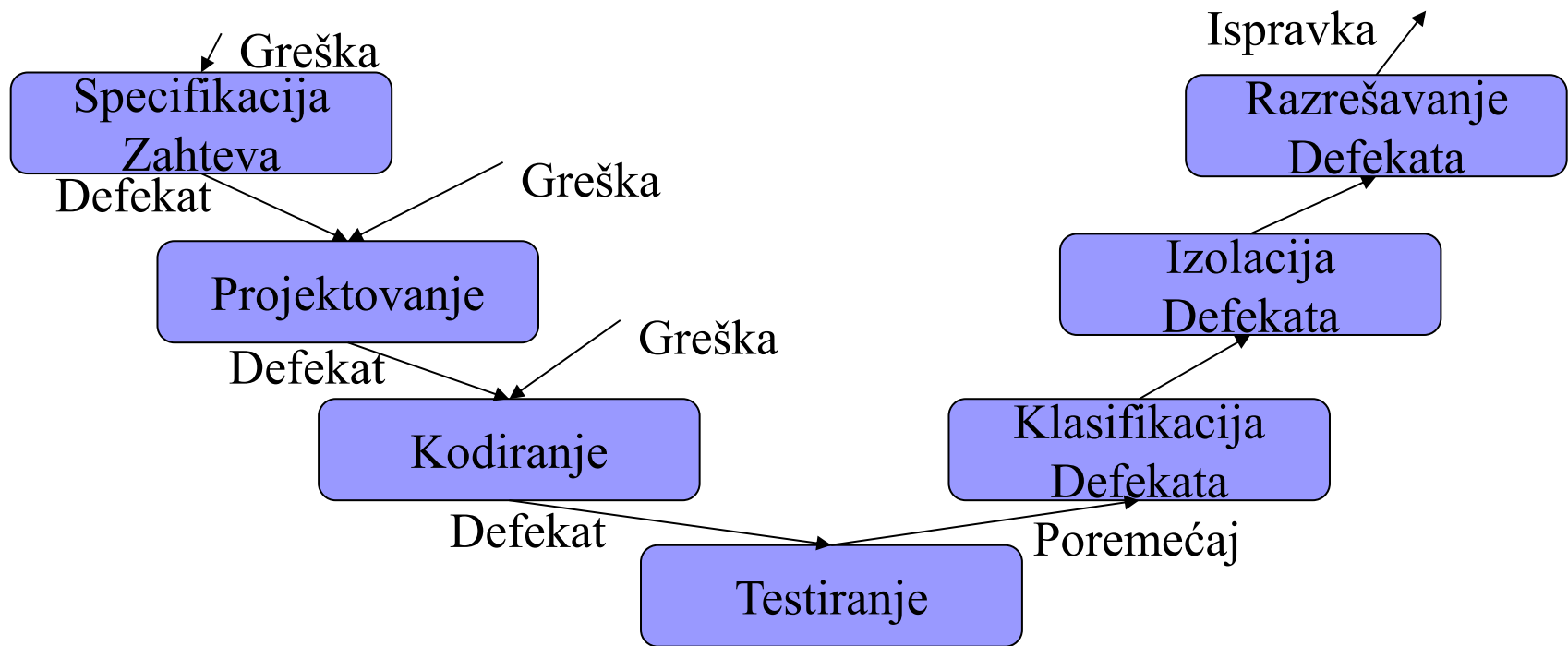
Setup, koji uključuje test drivere i ostale neophodne elemente, koji omogućavaju testiranje



Konceptualna šema testiranja



Mesto testiranja u razvoju softvera

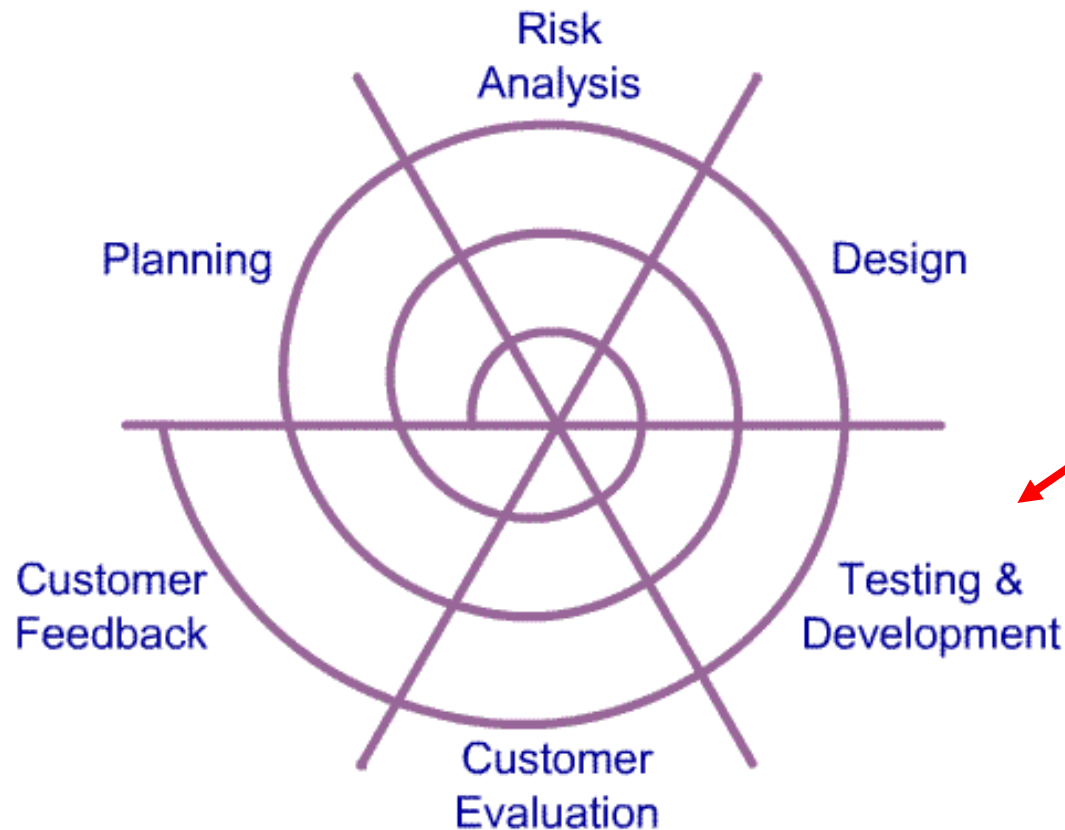


- Može se uočiti da u fazi razvoja postoje tri mesta gde može nastati greška, što rezultuje defektima koji se prenose u ostale razvojne faze.
- Ukratko, u prve tri faze bugovi se kreiraju, u fazi testiranja otkrivaju, a u poslednje tri faze rešavaju.
- Faza razrešavanja defekata pruža takođe priliku za nastajanje grešaka i novih defekata!!!



Mesto testiranja u razvoju softvera

Agile/spiralni model:



Zahtevi

■ funkcionalni:

- ☐ Ponašanje sistema treba da bude korektno
- ☐ Zahtevi mogu biti precizni ali često to nisu

■ ne-funkcionalni:

- ☐ performanse, pouzdanost, kompatibilnost, robusnost (stress/volume/recovery), ...
- ☐ Zahtevi mogu biti kvantitativni, i ne uvek odredjeni



Generisanje testova: Namena

Koje greške želimo da nadujemo?

Zavisi od faze u razvoju softvera:

- unit faza

Tipične typo greške (greške kodiranja), funkcionalni propusti

- integracija

Greške u interfejsima

- sistem/prijem: greške - zahtevi

- ☐ Neimplementirane funkcionalnosti

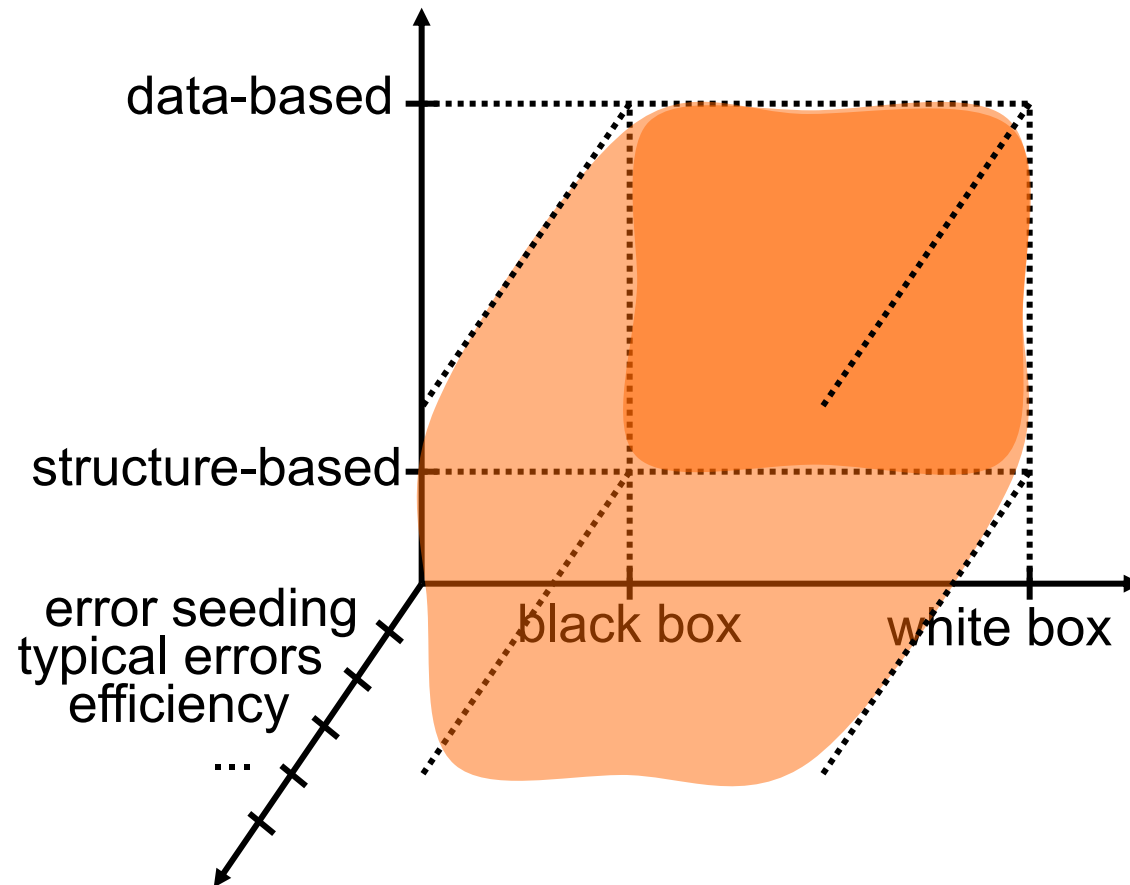
‘softver ne radi sve što treba da radi’

- ☐ Implementirane funkcionalnosti koje se ne zahtevaju



Generisanje testova: Tehnika

Dimenzije:



black box: nemamo pristup softveru (strukтури) koji testiramo

white box: imamo pristup softveru

Generisanje testova - Početak

Pretpostavke, ograničenja

- obične/višestrukе greške:

Objedinjavanje, medjusobna zavisnost grešaka

- Prethodne izmene

- Heuristike:

- ☐ Znanje o uobičajenim greškama i propustima!
- ☐ Istorija softvera (njegovog razvoja)
- ☐ Paradoks pesticida (Beizer 90: *“Every method you use to prevent or find bugs leaves a residue of subtler bugs against which those methods are ineffectual.”*)



Implementacija testova i izvršenje

■ Implementacija

- ☐ platforma
- ☐ batch
- ☐ ulazi, izlazi, upravljanje, ...

■ Izvršenje

- ☐ Aktuelno trajanje
- ☐ manuelno/interaktivno ili automatsko
- ☐ U paraleli na više sistema
- ☐ ponovljivo?

■ Alati/softveri za testiranje



Ko izvršava koji task (ko testira)?

- Realizatori softvera
 - ☐ programer
 - ☐ odeljenje za testiranje
- Korisnici softvera
 - ☐ krajnji korisnici
 - ☐ menadžment
- Ostali(Third party)
 - ☐ Testeri koji su eksterno angažovani
 - ☐ Sertifikacione organizacije

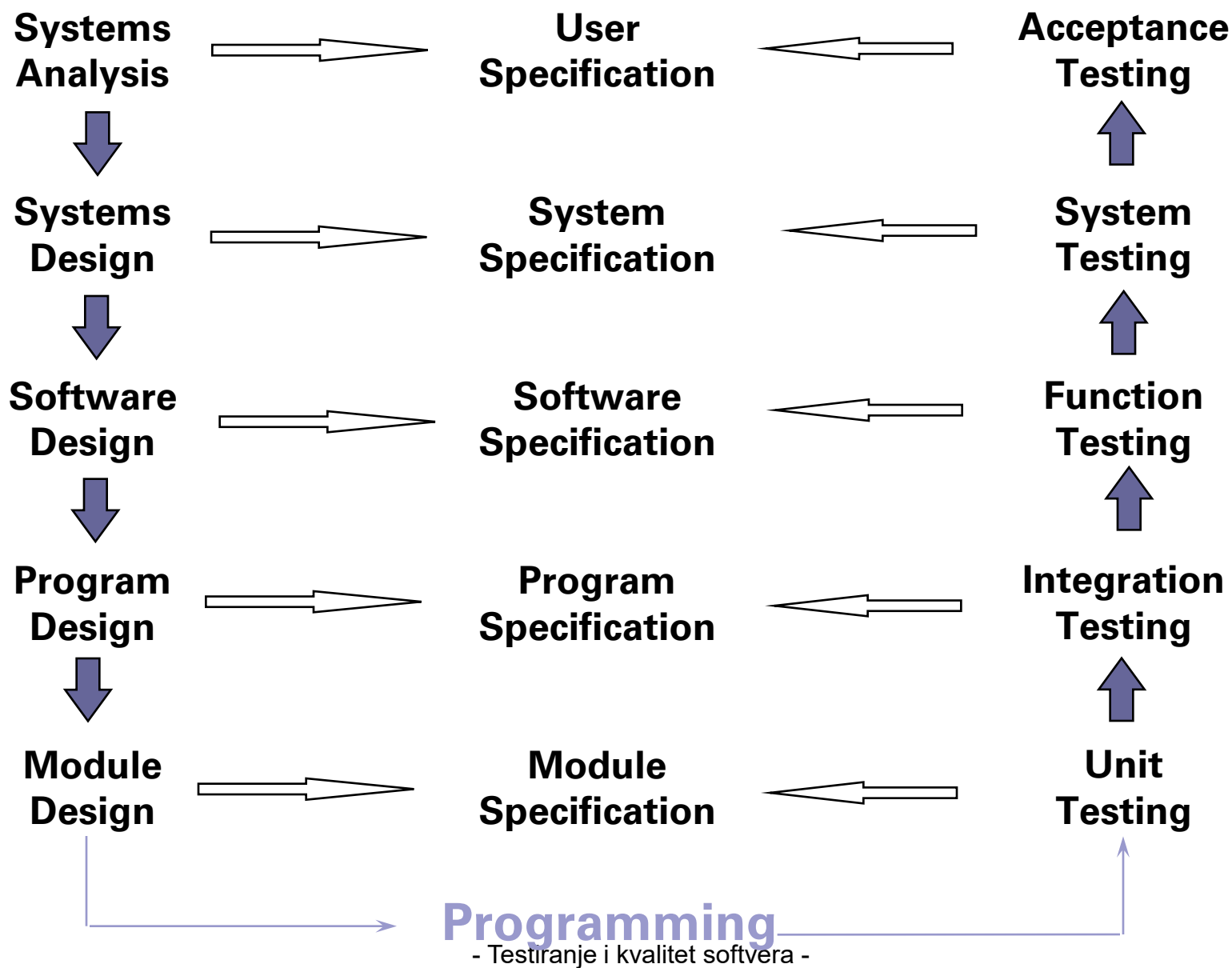


Evaluacija rezultata testiranja

- Za test:
 - ☐ pass/fail rezultat pojedinačnog testa
 - ☐ Dijagnostika izlaza
 - ☐ Koji zahtevi su bili a koji nisu bili ispunjeni
- Statističke informacije:
 - ☐ pokrivenost (programski kod, zahtevi, ulazni domen, izlazni domen)
 - ☐ progres testiranja (broj grešaka pronadjenih u jedinici vremena testiranja: smanjenje?)
- Odluke:
 - ☐ kraj (**satisfied**)
 - ☐ kreiraj/izvrši dodatne testove (**još nije dovoljno sigurno**)
 - ☐ podesi softver i/ili **zahteve**,
kreiraj/izvrši dodatne testove (**da se greška ispravi**)



Framework za testiranje IS



Klasifikacija testiranja (kriterijumi)

- Na osnovu cilja
- Po nameni
- Na osnovu oblasti
- Na osnovu faze u procesu razvoja softvera
- Na osnovu dostupnih informacija o softveru koji se testira



Klasifikacija na osnovu cilja

- Funkcionalno testiranje
- Testiranje performansi
- Stress (“load”) testiranje



Klasifikacija po nameni

- Testiranje usmereno na otkrivanje greške
- Prihvatno testiranje
- Regresiono testiranje
- Mutaciono testiranje



Klasifikacija po oblasti

- Testiranje delova (Unit testing)
- Integraciono testiranje (Integration testing)
- Testiranje sistema (System testing)



Klasifikacija po fazi u procesu razvoja softvera

- Unit testing: implementacija
- Integraciono testiranje: integracija podsistema
- Testiranje sistema: integracija sistema
- Prihvatno testiranje: deployment
- Regresiono testiranje: održavanje



Klasifikacija na osnovu dostupnih informacija

■ White-box testiranje ili

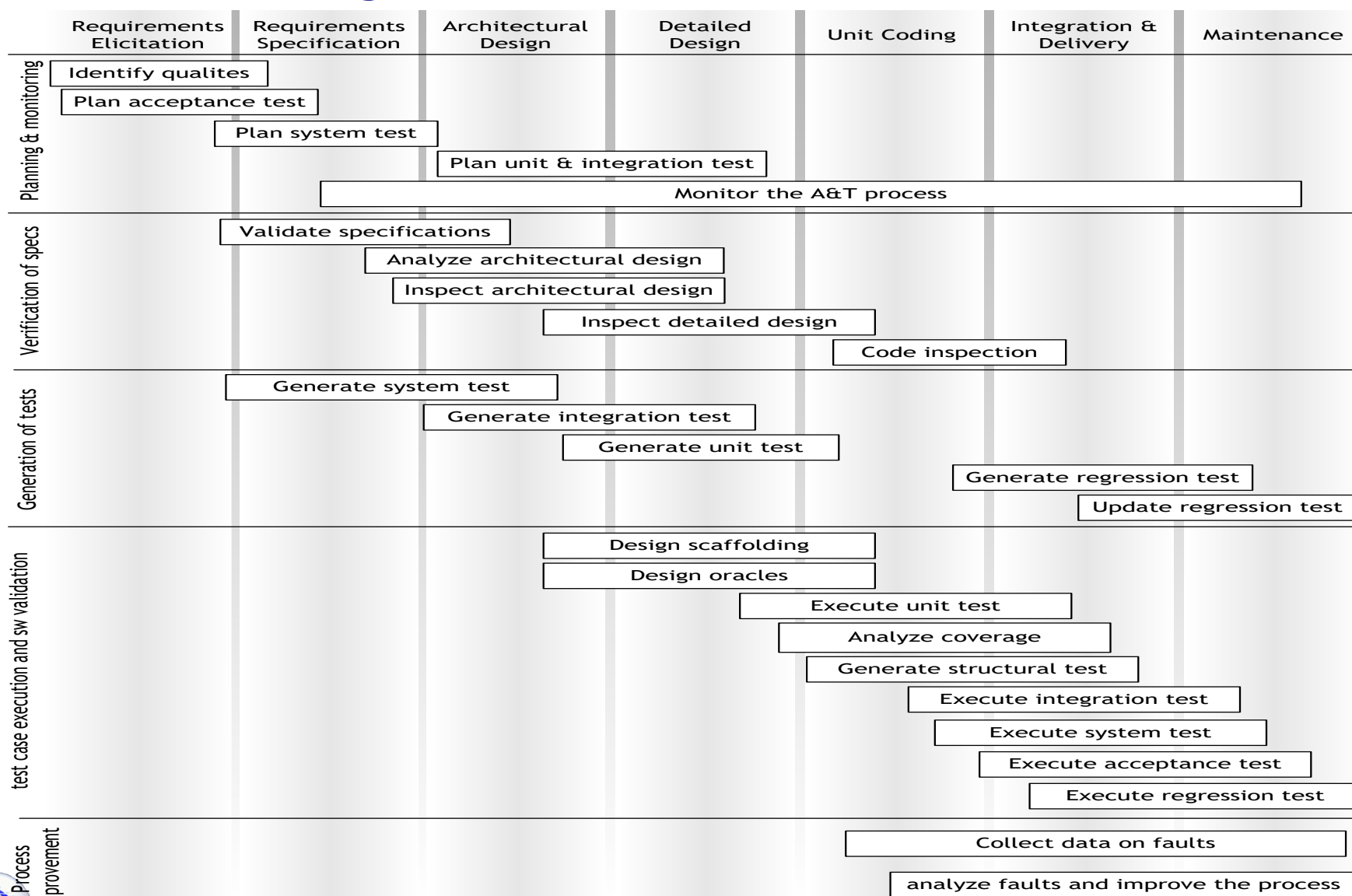
- ☐ implementation-based,
- ☐ structural,
- ☐ glass box,
- ☐ clear box.

■ Black-box testiranje ili

- ☐ responsibility-based,
- ☐ functional.



Testiranje - faze



Testiranje je teško: zašto?

- Ne postoji univerzalni postupak testiranja
- Vrste softverskih proizvoda
 - ☐ Desktop
 - ☐ Client- server
 - ☐ Distribuirani
 - ☐ Web
 - ☐ Objektno-orijentisani
 - ☐ Komponente
 - ☐ Service orijentisani
 - ☐ Mobilni softver
 - ☐ Cloud based
 - ☐ ...

■ Proces testiranja jako zavisi od vrste softvera!!!

Pitanja?

■ ?

■ ?

■ ?

